

Genetic Programming Classification

Table of Contents

1. Introduction	2
2. Dataset and Pre-processing	3
2.1. Dataset Overview	3
2.2. Data Preprocessing	3
3. Methodology	4
3.1. Population Initialisation	4
3.2. Fitness Evaluation	4
3.3. Selection	5
3.4. Crossover	5
3.5. Mutation	5
3.6. Depth Control and Bloat Prevention.....	6
3.7. Elitism and Generational Replacement.....	6
3.8. Arithmetic GP Classifier.....	6
3.9. Decision Tree GP Classifier	7
4. Experimental Setup	8
4.1. Parameters	8
4.2. Evaluation Metrics.....	8
4.2.1. Accuracy	8
4.2.2. F-Measure (F1-Score).....	9
4.2.3. Runtime	9
4.3. Experimental Protocol	9
4.4. Statistical Significance Testing	9
5. Results	Error! Bookmark not defined.

1. Introduction

Breast cancer recurrence prediction is an important classification problem in medical data analysis, as early and accurate predictions can improve treatment decisions and patient outcomes. Machine learning and evolutionary computation techniques have increasingly been applied to medical datasets to develop predictive models with high accuracy and interpretability.

This report investigates the use of Genetic Programming (GP) for the binary classification of breast cancer recurrence using the Wisconsin Breast Cancer dataset. Two GP approaches are implemented and compared: an Arithmetic GP classifier that evolves mathematical expressions using arithmetic operators, and a Decision Tree GP classifier that evolves logical decision structures using conditional and comparison operators.

Both models are evaluated over 30 independent runs using performance metrics including training accuracy, test accuracy, F1-score, and runtime. The study further compares the effectiveness, interpretability, and statistical significance of the two approaches in solving the classification problem.

2. Dataset and Pre-processing

2.1. Dataset Overview

The dataset used in this study is the Wisconsin Breast Cancer dataset provided for the assignment as two pre-encoded CSV files: `Breast_train.csv` and `Breast_test.csv`. The dataset contains patient records describing tumour characteristics and demographic information, intending to predict whether breast cancer will recur.

The classification task is binary, where:

- 0 represents no recurrence of breast cancer
- 1 represents recurrence of breast cancer

The dataset contains nine input features related to patient demographics and tumour characteristics, including age, menopause status, tumour size, involved lymph nodes, node capsule status, degree of malignancy, breast side, breast quadrant, and irradiation status. All categorical attributes were already numerically encoded in the provided dataset.

2.2. Data Preprocessing

Minimal preprocessing was required because the dataset was already numerically encoded. The data was loaded directly into the Genetic Programming framework using a custom CSV parsing class (`CSVLoader.java`).

The preprocessing steps performed were as follows:

- The CSV files were loaded while skipping the header row.
- Feature values were stored as numeric (double) values.
- Class labels were stored as integer values (0 or 1).
- Missing values encoded within the dataset were retained as valid categorical values.
- No normalisation or feature scaling was applied, as Genetic Programming operates effectively on raw symbolic feature values.

The provided dataset already contained separate training and testing files, ensuring consistent evaluation across all experimental runs.

3. Methodology

This section describes the complete GP system design: the shared Core Engine used by both models, the Arithmetic GP Classifier, and the Decision Tree GP Classifier. All components were implemented in Java. The Core Engine was built as a reusable framework so that both models plug into the same evolutionary infrastructure with only their function sets and terminal sets differing.

3.1. Population Initialisation

The initial population of 200 individuals is generated using the Ramped Half-and-Half method. This technique divides the population evenly across initial tree depths ranging from 2 to 6. For each target depth, half the trees are built using the Full method (all branches reach the maximum depth; only terminals appear at leaves) and the other half using the Grow method (branches may terminate early at any depth up to the maximum). The combined effect is a diverse initial population with varying sizes and shapes, reducing the risk of premature convergence.

The `growTree` method builds trees recursively: at the maximum depth, it forces a terminal node; under the Grow method, the probability of selecting a terminal increase as the current depth increases; under the Full method, function nodes are always selected until the maximum depth is reached, then terminals are forced.

3.2. Fitness Evaluation

Fitness is defined as classification accuracy on the training set, the proportion of instances that the evolved program classifies correctly:

$$\text{fitness} = \frac{\text{(number of correctly classified instances)}}{\text{(total training instances)}}$$

Higher accuracy is better (maximisation problem). For the Arithmetic GP, the tree produces a real-valued output.

A threshold of 0.5 is applied:

- $\text{output} \geq 0.5$ predicts class 1 (recurrence)
- $\text{output} < 0.5$ predicts class 0.

For the Decision Tree GP, the `IfThenElseNode` structure and `ClassNode` leaves produce 0 or 1 directly, so no thresholding is required.

3.3. Selection

Tournament selection is used with a tournament size of 7 to select parent individuals for reproduction.

- Randomly select 7 individuals from the population
- Compare the fitness values of the selected individuals
- Return a deep copy of the individual with the highest fitness
- Repeat the process for each parent selection event

This method maintains selection pressure while reducing the risk of premature convergence.

3.4. Crossover

Subtree crossover is applied with a probability of 85% to generate offspring from two selected parent trees.

- Select two parent individuals using tournament selection
- Choose a random crossover point in each parent tree
- Swap the subtrees rooted at the selected nodes
- Verify that both offspring satisfy the maximum depth constraint ($\text{maxDepth} = 8$)
- If the depth constraint is violated, retry using different crossover points (up to 20 attempts)
- If no valid crossover is found, return copies of the original parents unchanged

The implementation ensures that the root node is never swapped (to maintain tree validity).

3.5. Mutation

Point mutation is applied with a probability of 15%. A single node is selected uniformly at random from the tree. The replacement rule depends on the node's type:

- Terminal node (arity 0): replaced with a randomly chosen terminal from the appropriate terminal set.
- Function node (arity > 0): replaced with a randomly chosen function of the same arity from the appropriate function set, preserving all existing child subtrees.

The same-arity requirement is critical: it ensures that the structural integrity of the subtree below the mutated node is maintained. After mutation, a depth check is applied; if the maximum depth of 8 is violated, up to 20 retries are attempted. If no valid mutation is found, the original tree is returned unchanged.

3.6. Depth Control and Bloat Prevention

Without explicit depth control, GP populations are prone to bloat, the uncontrolled growth of tree size across generations without a corresponding improvement in fitness. The following mechanisms are enforced throughout the evolutionary cycle:

- Maximum offspring depth of 8 is applied after every crossover and mutation operation.
- Initial tree depths bounded between 2 and 6 during Ramped Half-and-Half initialisation.
- A retry mechanism (maximum 20 attempts) is used for both crossover and mutation before reverting to the parent(s).

This combination of hard depth limits and retry-based enforcement ensures that tree size remains manageable across 100 generations, keeping evaluation times predictable.

3.7. Elitism and Generational Replacement

The top 2 individuals (by fitness) from each generation are copied unchanged into the next generation. This guarantees that the best fitness found so far is never lost due to stochastic variation in crossover or mutation. The remaining 198 positions in the new population are filled by offspring produced through tournament selection and crossover/mutation.

3.8. Arithmetic GP Classifier

The Arithmetic GP evolves real-valued expression trees. Each individual computes a numeric score from the nine input features. Classification is performed by comparing the tree's output to a threshold of 0.5: if the output is ≥ 0.5 , the instance is assigned to class 1 (recurrence); otherwise class 0.

Component	Details
Function set	$\{+, -, \times, \div\}$ protected division returns 1.0 when $ \text{denominator} < 1 \times 10^{-6}$.

Terminal set	9 FeatureNodes ($f[0] - f[8]$) + ConstNodes with integer values in $\{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$.
Classification rule	output $\geq 0.5 \rightarrow$ class 1 (recurrence); output $< 0.5 \rightarrow$ class 0.

Using a constant set that spans both negative and positive values gives the arithmetic tree sufficient expressive power to construct classifiers that can partition the feature space with both additive and subtractive expressions, without requiring continuous random constants.

3.9. Decision Tree GP Classifier

The Decision Tree GP evolves logical decision programs. Rather than producing a numeric score, the tree structure itself encodes a decision procedure: IfThenElseNode branches route each instance through comparison and logical conditions until a leaf ClassNode is reached, returning 0 or 1 directly. No post-hoc thresholding is needed.

Component	Details
Function set	$\{ IF - THEN - ELSE \text{ (arity 3)}, <, >, == \text{ (epsilon } 1 \times 10^{-6}), AND, OR, NOT \}$.
Terminal set	9 FeatureNodes ($f[0] - f[8]$) + ClassNode(0) + ClassNode(1) + ConstNodes $\{ 0, 1, 2, 3, 4, 5 \}$.
Classification rule	The tree directly returns 0 or 1. No threshold applied.
Equality epsilon	1×10^{-6} ensures robust floating-point equality checks on integer-encoded categorical features.

The inclusion of constants 0–5 in the terminal set allows comparison nodes (e.g., LessThanNode, GreaterThanNode) to test individual feature values directly against meaningful category boundaries, as all features are encoded as small non-negative integers. The NOT operator allows the tree to express negated conditions without requiring duplicate subtrees.

4. Experimental Setup

This section describes the hyperparameter configuration, evaluation metrics, and experimental protocol followed for both GP models. All parameters were held constant across both models to ensure a controlled and fair comparison.

4.1. Parameters

Parameter	Arithmetic GP	Decision Tree GP
Population size	200	200
Maximum generations	100	100
Initialisation method	Ramped Half-and-Half	Ramped Half-and-Half
Initial depth range	2 – 6	2 – 6
Maximum offspring depth	8	8
Selection method	Tournament	Tournament
Tournament size	7	7
Crossover probability	85%	85%
Mutation probability	15%	15%
Mutation type	Point (same arity)	Point (same arity)
Elitism count	2	2
Max retries (depth violations)	20	20
Fitness function	Accuracy	Accuracy
Independent runs	30	30

4.2. Evaluation Metrics

Three metrics are reported for both models, consistent with the assignment specification:

4.2.1. Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Reported separately for the training set (to measure model fit) and the test set (to measure generalisation).

4.2.2. F-Measure (F1-Score)

The F1-score is computed for the positive class (class 1: recurrence), which is typically the minority class in this dataset:

$$\begin{aligned}\text{Precision} &= \text{TP} / (\text{TP} + \text{FP}) \\ \text{Recall} &= \text{TP} / (\text{TP} + \text{FN}) \\ \text{F1} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})\end{aligned}$$

If $\text{TP} + \text{FP} = 0$ or $\text{TP} + \text{FN} = 0$, the corresponding metric is set to 0 to avoid division by zero.

4.2.3. Runtime

Execution time is measured from the start of the evolutionary loop to the end of the final generation, in milliseconds. Data loading and post-processing are excluded. Runtime reflects the computational cost of the evolutionary process alone and is used to compare the two models' efficiency.

4.3. Experimental Protocol

To ensure statistical significance and reproducibility:

- **30 independent runs** are performed for each model type
- Each run uses a unique seed: $\text{seed} = \text{run_index} \times 12345L$
- Each run produces: final training accuracy, test accuracy, F1-score, and runtime
- Aggregate statistics: mean and standard deviation for each metric
- Best model (highest test accuracy) is saved for each model type
- Statistical significance testing is performed using [paired t-test / Wilcoxon signed-rank test]

4.4. Statistical Significance Testing

A statistical significance test is applied to the 30 test accuracy values from each model to determine whether the observed difference in performance is meaningful or could be attributed to random variation. A Wilcoxon Signed-Rank Test (or paired T-test, if normality is assumed) is used on the paired samples (same 30 seeds applied to both models). The null hypothesis (H_0) is that there is no difference in test accuracy between the two models; the significance level is $\alpha = 0.05$.